Morphological Image Processing: A Sequential and OpenMP Parallelized Approach

Francesco Stocchi
francesco.stocchi1@edu.unifi.it

Abstract

This project explores the implementation and performance analysis of fundamental morphological image processing operators (erosion, dilation, opening, and closing) in C++, with a sequential baseline and three progressively refined OpenMP-parallelized versions. The parallel variants differ in how image borders are handled: a naive parallelization of the sequential code, a version that separates border and interior pixel processing to remove branching from the hot loop, and a version that additionally parallelizes the border computation itself using OpenMP tasks. A dedicated pipeline based on MS COCO 2017 images was built to generate benchmark datasets of increasing resolution. Results demonstrate that removing conditional branching from the parallel region has a far greater impact on speedup and efficiency than the specific scheduling strategy used for the image borders. Link to the repository

1. Introduction

1.1. Understanding Morphological Image Processing

Mathematical morphology is a theory for analyzing and processing geometrical structures in images, originally developed for binary images and later extended to grayscale. The two fundamental operators, **erosion** and **dilation**, probe an image with a small shape called the *structuring element*: erosion shrinks the foreground regions of an image by requiring the structuring element to fit entirely within them, while dilation grows foreground regions by marking any pixel touched by the structuring element. From these two primitives, more complex operators are derived by composition: **opening** (erosion followed by dilation) removes small bright details and separates weakly connected objects, while **closing** (dilation followed by erosion) fills small dark gaps and holes.

Because every output pixel depends only on a local neighborhood of the input image defined by the structuring element, morphological operators are naturally data-parallel: the computation for one pixel is independent of all others, making this class of algorithms a good candidate for shared-memory parallelization with OpenMP. However, this same locality also introduces a practical complication at implementation time: pixels near the image border have an incomplete neighborhood, which must be handled explicitly (e.g. via clamping) and, if implemented naively, introduces branching that can hurt both sequential and parallel performance.

1.2. Related Work

This project builds on the standard formulation of grayscale morphological operators as they are commonly presented in image processing literature, and reuses course material and structuring element conventions introduced in the Parallel Programming course. Rather than modifying the algorithm itself, the project focuses on how different low-level implementation choices for handling image borders affect the effectiveness of OpenMP-based parallelization on a compute-bound, memory-regular kernel.

2. Language and Libraries

The project is implemented in C++ and uses OpenMP for shared-memory parallelization of the pixel-processing loops. OpenMP was chosen because morphological operators map naturally onto simple `parallel for` loops over image rows/columns, and because it allows incrementally introducing more advanced constructs (such as tasks) without restructuring the underlying algorithm.

Image loading and decoding is handled through the `stb_image` single-header library, kept lightweight and dependency-free to simplify benchmarking across machines. A custom `save_results_csv` utility writes benchmark results (timing, speedup, dataset, thread count, variant) to CSV files for later analysis. Dataset preparation is performed offline with a Python pipeline (`Automation.py`) that downloads and preprocesses source images before the C++ benchmarks are run.

3. Methodology

3.1. Sequential Baseline

The sequential implementation (`morphology_seq.cpp/h`) provides `erosion_seq` and `dilation_seq`, from which `opening_seq` (erosion followed by dilation) and `closing_seq` (dilation followed by erosion) are composed. A square structuring element of side k is used ($k = 5$ in all experiments reported here), and border pixels are handled with a conditional check at every pixel of the loop to determine whether the structuring element window falls fully inside the image or needs clamping. This naive border handling, while simple and correct, places a branch inside the innermost loop for every single pixel, including the vast majority that are far from any border. A representative pseudocode of the erosion kernel is given in the Appendix (Listing 1).

3.2. Parallel Implementation: Three Variants

Three progressively refined OpenMP parallelizations (`morphology_omp.cpp/h`) were implemented and benchmarked against the sequential baseline, in order to isolate the impact of border handling on parallel efficiency.

3.2.1. Variant 1 – Naive Parallelization

.

The first variant (`_omp`) applies `#pragma omp parallel for collapse(2)` directly on top of the unmodified sequential loop structure. The per-pixel border check is still present inside the hot loop executed by every thread. This variant isolates the raw benefit of multithreading alone, without addressing the branching overhead inherited from the sequential version.

3.2.2. Variant 2 – Border/Interior Separation

.

The second variant (`_omp_1`) restructures the computation by explicitly separating the image into two regions: a thin border region, whose pixels are computed serially with explicit clamping, and the interior region, which is processed in parallel with `#pragma omp parallel for` using direct, unchecked access to contiguous pointers – no bound-checking branch is present in this hot loop. Because the interior of realistic images vastly outnumbers the border pixels, this restructuring removes the per-pixel branch from almost the entire parallel workload while keeping the (cheap, small) border computation simple and correct.

3.2.3. Variant 3 – Parallel Border via OpenMP Tasks

.

The third variant (`_omp_2`) starts from the same border/interior split as Variant 2, but additionally parallelizes the four image borders using OpenMP tasks (one task per border), before processing the interior with a standard `parallel for`. The goal of this variant was to test whether parallelizing the (small) border computation could further reduce total execution time, or whether the fixed overhead of task creation and synchronization would outweigh the limited amount of work available on the borders.

A simplified comparison of the border-handling strategy for Variants 2 and 3 is provided in the Appendix (Listing 4).

3.3. Correctness Verification

Before any timing measurement is trusted, a dedicated `BenchmarkRunner` component validates correctness for every dataset and core-count configuration: it performs a pixel-perfect comparison between the sequential output and each of the three parallel variants, and additionally cross-checks the parallel variants against one another (V1 vs. V2, V1 vs. V3), so that a regression introduced by one variant cannot be masked by an independent bug in another. If any mismatch is detected, the offending image pair is written to disk for inspection rather than silently discarded. All configurations used in Section 4 passed this pixel-perfect check against the sequential baseline.

3.4. Benchmark Dataset Generation

To evaluate the implementations across a range of image sizes, a dedicated Python pipeline (`Automation.py` + `config.json`) was built to generate four benchmark datasets of increasing resolution (*small*, *medium*, *large*, *xlarge*). The pipeline downloads $20{,}000$ source images from the MS COCO 2017 panoptic annotations (retrieved via Archive.org), groups them by original size, and redistributes them across the four target datasets using a load-balancing criterion proportional to $1/\text{target\_size}^2$, so that datasets with larger target resolutions contain proportionally fewer images, keeping the total computational cost of each dataset comparable. Selected images are then resized with bicubic interpolation to 512, 1024, 2048, and 4096 pixels respectively, and converted to grayscale, since the morphological operators under study operate on single-channel images.

4. Results and Discussion

4.1. Experimental Setup

All benchmarks were run on a machine with 20 physical cores. The thread count was swept from 1 (serial baseline) to 19 (max-1 cores, leaving one core free for the OS), for each of the three parallel variants, each of the four morphological operators (erosion, dilation, opening, closing), and each of the four benchmark datasets (*small*, 512px; *medium*, 1024px; *large*, 2048px; *xlarge*, 4096px), with 50 images per dataset. Reported times are averages over multiple repeated runs per configuration. The average sequential execution time per dataset ranges from 0.20 s (*small*) to 10.6 s (*xlarge*); only the *xlarge* dataset satisfies the $\geq$ 10 s rule of thumb for reliable timing, which is kept in mind when interpreting the noisier curves for the smaller datasets in Section 4.2. Before each parallel configuration is timed, a warmup pass is run on a 512×512 image to initialize the OpenMP thread pool and warm the cache, avoiding cold-start bias in the measurements.

4.2. Comparison of Parallel Variants

Figure 1 shows speedup versus thread count for the three parallel variants, averaged over the four morphological operators, for each of the four datasets; Figure 2 shows the corresponding parallel efficiency ($\mathrm{Speedup}(p)/p$). The three variants separate clearly and consistently across all dataset sizes:

- **Variant 1 (naive)** plateaus early and never exceeds a speedup of about 2.7×, with efficiency collapsing to 0.09–0.15 at high thread counts. The per-pixel border branch retained in the hot loop limits scalability regardless of how many threads are available.
- **Variant 2 (border/interior split)** is the best-performing variant on every dataset, reaching speedups between 13.1× and 15.1× (efficiency 0.74–1.05) at its optimum, typically around 13–18 threads.
- **Variant 3 (parallel border via tasks)** tracks Variant 2 closely only at low thread counts, then degrades steadily as thread count increases, dropping *below* the serial baseline (speedup < 1) from as few as 8 threads on the *large* and *xlarge* datasets, and from 13–15 threads on the smaller ones.

4.3. Analysis of the Best-Performing Variant

Figure 3 isolates Variant 2 across all four dataset sizes. Two effects are visible. First, at very low thread counts (2–4 threads) Variant 2 exhibits *superlinear* speedup, with efficiency up to 2.9× the ideal value: splitting the interior

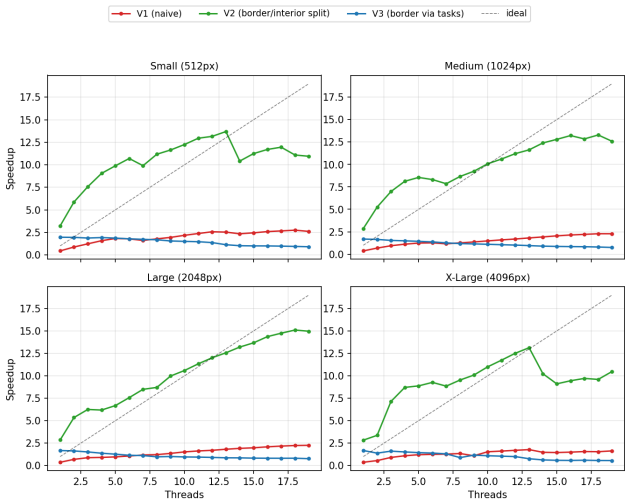

Figure 1: Speedup vs. thread count for the three parallel variants, averaged over the four morphological operators, per dataset. Dashed line marks ideal (linear) speedup.

into a small number of large, contiguous chunks appears to improve cache locality relative to the single-threaded baseline, an effect that fades as the chunks shrink with more threads. Second, speedup saturates and mildly declines beyond roughly 15–18 threads, i.e. as the run approaches the machine's 20-core limit: with fewer and fewer image rows left per thread, per-thread overhead (scheduling, cache refills at chunk boundaries) starts to outweigh the shrinking amount of useful work each thread is given, yielding diminishing returns rather than further gains.

The gap between Variant 1 and Variant 2 confirms that, for this kernel, *removing branching from the hot loop* is far more impactful than the raw act of adding `parallel for`: both variants parallelize the same amount of work, but only Variant 2 gives every thread a branch-free, contiguous-memory inner loop. Variant 3's degradation with increasing thread count supports the overhead hypothesis formed from the preliminary data: the image border is a fixed, small amount of work (four thin strips), so wrapping it in four OpenMP tasks adds a roughly constant task-creation/synchronization overhead that does not shrink as more threads become available, while the interior-loop benefit stays the same as in Variant 2. As thread count grows, this fixed overhead becomes an increasingly large fraction of the (shrinking) per-thread interior work, eventually making Variant 3 slower than not parallelizing the border at all.

5. Conclusion

5.1. Key Results

Across all four datasets, separating border and interior pixel processing (Variant 2) was consistently the best-

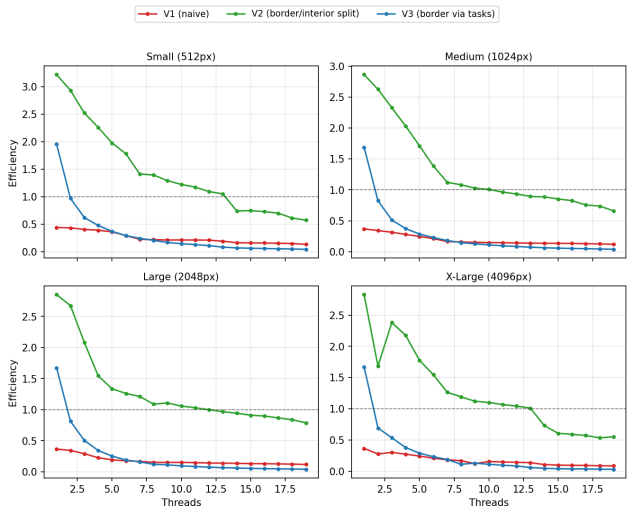

Figure 2: Parallel efficiency vs. thread count for the three parallel variants, averaged over the four morphological operators, per dataset. Dashed line marks ideal efficiency ($= 1$).

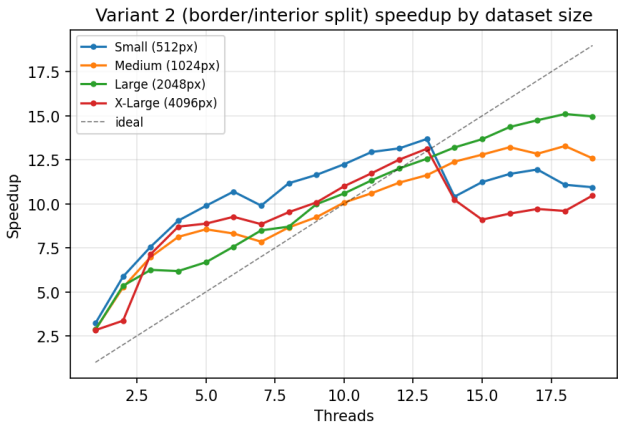

Figure 3: Speedup of Variant 2 (border/interior split) across the four dataset sizes.

performing parallelization strategy, achieving speedups of $13.1\times$–$15.1\times$ and near-ideal efficiency (0.74–1.05) at its optimal thread count (typically 13–18 threads on a 20-core machine). The naive parallelization (Variant 1), which kept the original per-pixel border branch inside the hot loop, was limited to $1.8\times$–$2.7\times$ speedup regardless of thread count. Adding task-based parallelism for the (small) border computation (Variant 3) provided no benefit over Variant 2 and actively hurt performance at higher thread counts, eventually running slower than the sequential baseline.

5.2. Implications

The central takeaway is that, for compute-bound, memory-regular kernels such as morphological operators, *how* a loop is parallelized matters more than *how many* threads are thrown at it: eliminating a per-pixel conditional branch from the hot loop (Variant 2) yielded roughly a 5–$6\times$ larger speedup than naive parallelization of the exact same workload (Variant 1). By contrast, applying additional parallel constructs (OpenMP tasks) to an already small, fixed amount of work (Variant 3) is not "free" – its fixed overhead can outweigh its benefit once enough threads are already saturating the main workload, turning an intended optimization into a regression. More generally, this suggests that parallelization effort is best directed at removing overhead from the *dominant* portion of a kernel's work, rather than at parallelizing every individual code region regardless of its size.

References

[1] R. C. Gonzalez, R. E. Woods. *Digital Image Processing*. Pearson, 4th Edition, 2018.

[2] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Specification*. Version 5.2, 2021.

[3] T.-Y. Lin et al. *Microsoft COCO: Common Objects in Context*. ECCV, 2014.

[4] S. Barrett. *stb_image – single-file public domain image loader*. https://github.com/nothings/stb.

[5] Marco Bertini. *Parallel Programming Course Lectures*. Department of Information Engineering, University of Florence, 2025.

A. Appendix

A.1. Sequential Kernels for the Four Operators (Pseudocode)

Listing 1: Sequential erosion with naive per-pixel border check

```
for (int y = 0; y < H; y++) {
  for (int x = 0; x < W; x++) {
    uint8_t min_val = 255;
    for (int dy = -k/2; dy <= k/2; dy++) {
      for (int dx = -k/2; dx <= k/2; dx++) {
        int ny = y + dy, nx = x + dx;
        if (ny >= 0 && ny < H && nx >= 0 && nx < W)
            {
          min_val = std::min(min_val, in[ny*W+nx]);
        } // else: implicit clamp, skip
      }
    }
    out[y*W+x] = min_val;
  }
}
```

Listing 2: Sequential dilation – mirror of erosion, tracking the maximum instead of the minimum

```
for (int y = 0; y < H; y++) {
```

```
  for (int x = 0; x < W; x++) {
    uint8_t max_val = 0;
    for (int dy = -k/2; dy <= k/2; dy++) {
      for (int dx = -k/2; dx <= k/2; dx++) {
        int ny = y + dy, nx = x + dx;
        if (ny >= 0 && ny < H && nx >= 0 && nx < W)
            {
          max_val = std::max(max_val, in[ny*W+nx]);
        } // else: implicit clamp, skip
      }
    }
    out[y*W+x] = max_val;
  }
}
```

Listing 3: Opening and closing, composed from erosion_seq/dilation_seq via an intermediate buffer

```
// Opening: erosion followed by dilation
void opening_seq(in, out, temp, W, H, k) {
  erosion_seq(in, temp, W, H, k);
  dilation_seq(temp, out, W, H, k);
}

// Closing: dilation followed by erosion
void closing_seq(in, out, temp, W, H, k) {
  dilation_seq(in, temp, W, H, k);
  erosion_seq(temp, out, W, H, k);
}
```

A.2. Border/Interior Split (Variants 2 and 3)

Listing 4: Interior loop with no bound-checking branch (Variant 2); Variant 3 wraps the four border loops in OpenMP tasks before this point

```
// Borders (thin strips) computed serially with
    clamping
compute_border_top(in, out, W, H, k);
compute_border_bottom(in, out, W, H, k);
compute_border_left(in, out, W, H, k);
compute_border_right(in, out, W, H, k);

// Interior: unchecked, contiguous pointer access
#pragma omp parallel for collapse(2)
for (int y = k/2; y < H - k/2; y++) {
  for (int x = k/2; x < W - k/2; x++) {
    uint8_t min_val = 255;
    for (int dy = -k/2; dy <= k/2; dy++) {
      const uint8_t* row = in + (y+dy)*W + x - k
          /2;
      for (int dx = 0; dx < k; dx++) {
        min_val = std::min(min_val, row[dx]);
      }
    }
    out[y*W+x] = min_val;
  }
}
```

A.3. Contribution Statement

1. This is a course project for Parallel Programming, University of Florence; the computer vision component (morphological operators) is standard and used here as a vehicle for studying parallelization strategies.
2. All ideas, algorithm formulation, software implementation, experimental design, and analysis in this report are the author's own work.
3. The author confirms being the sole author of this writeup.